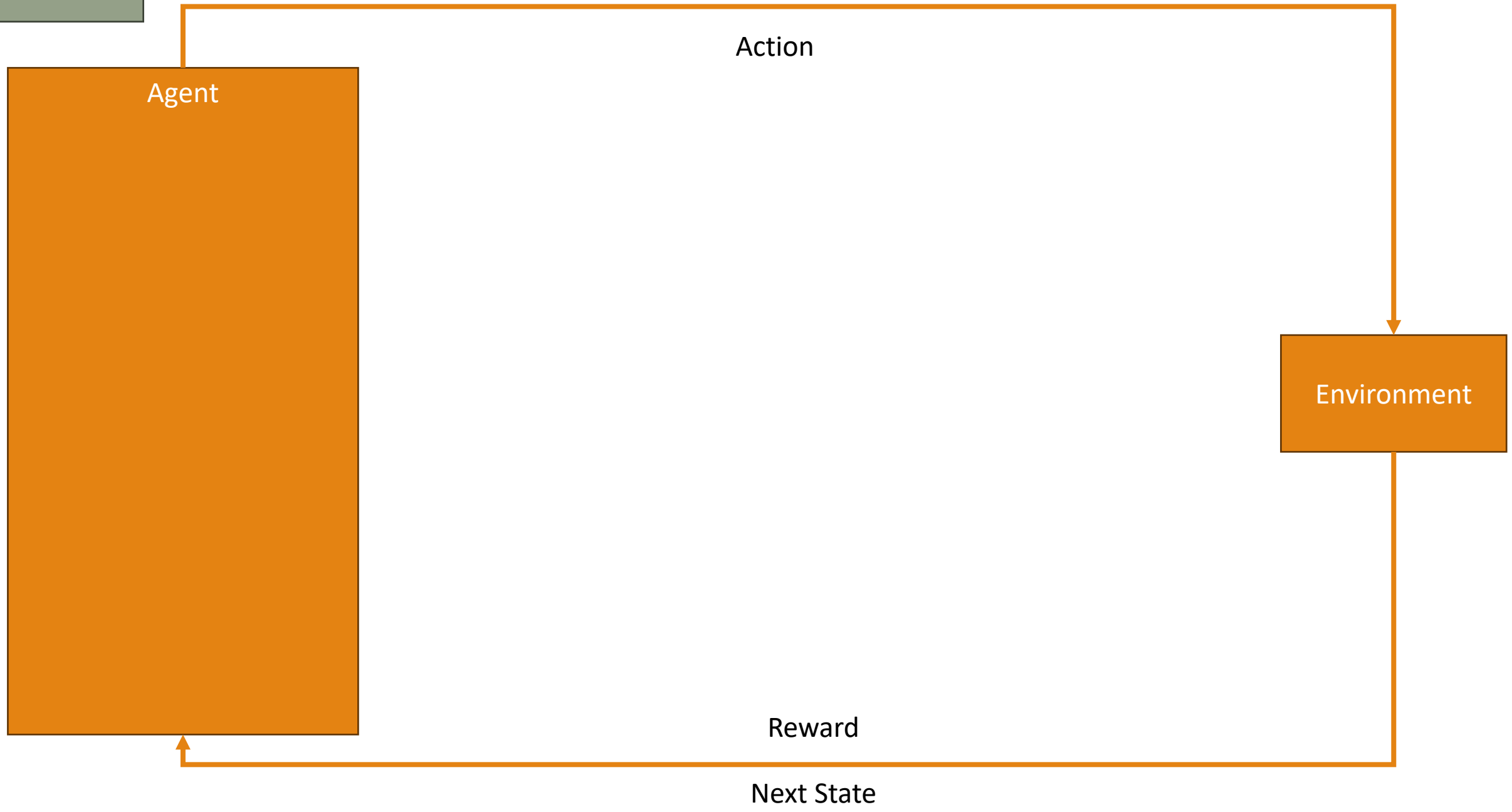
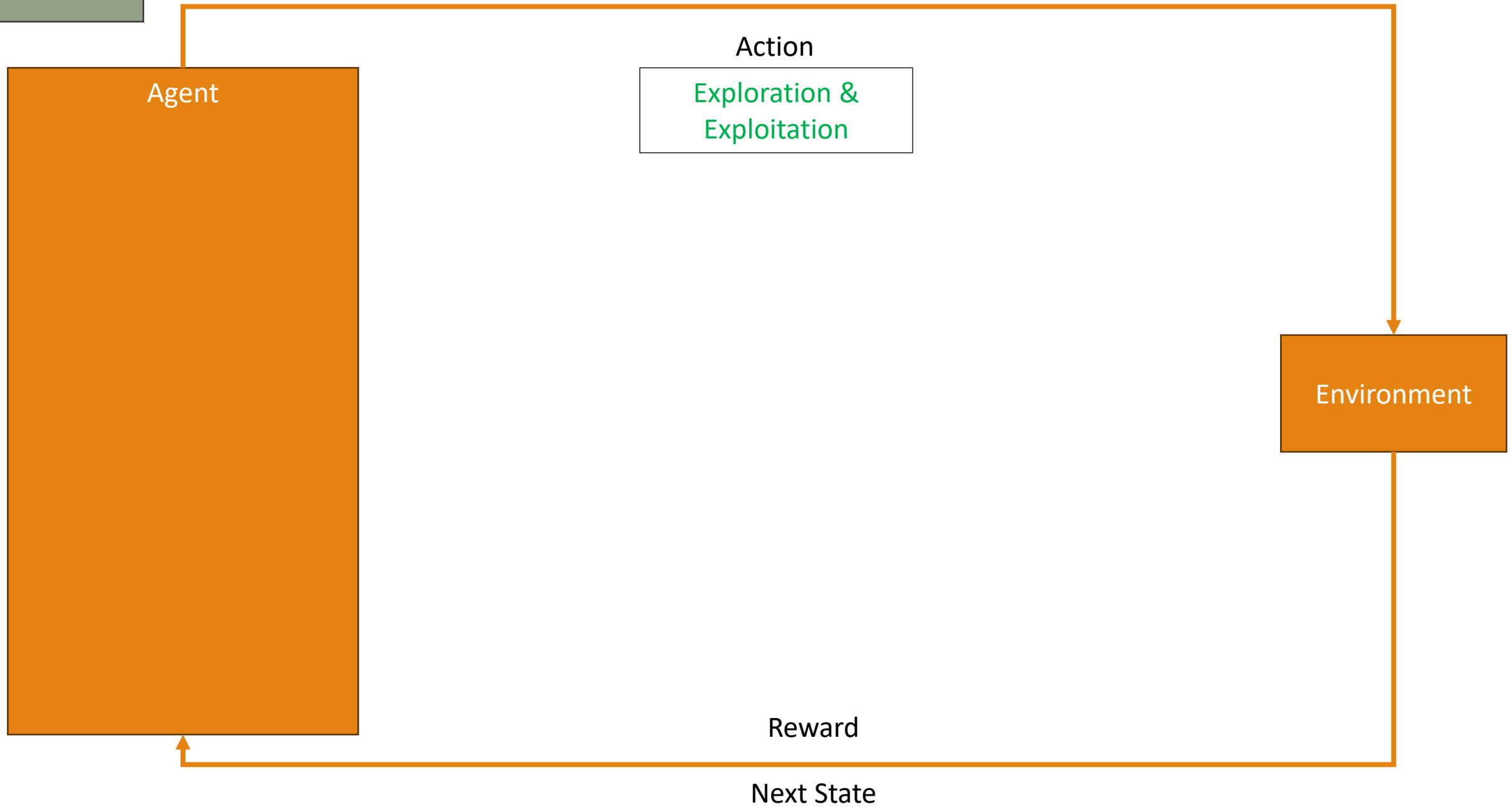
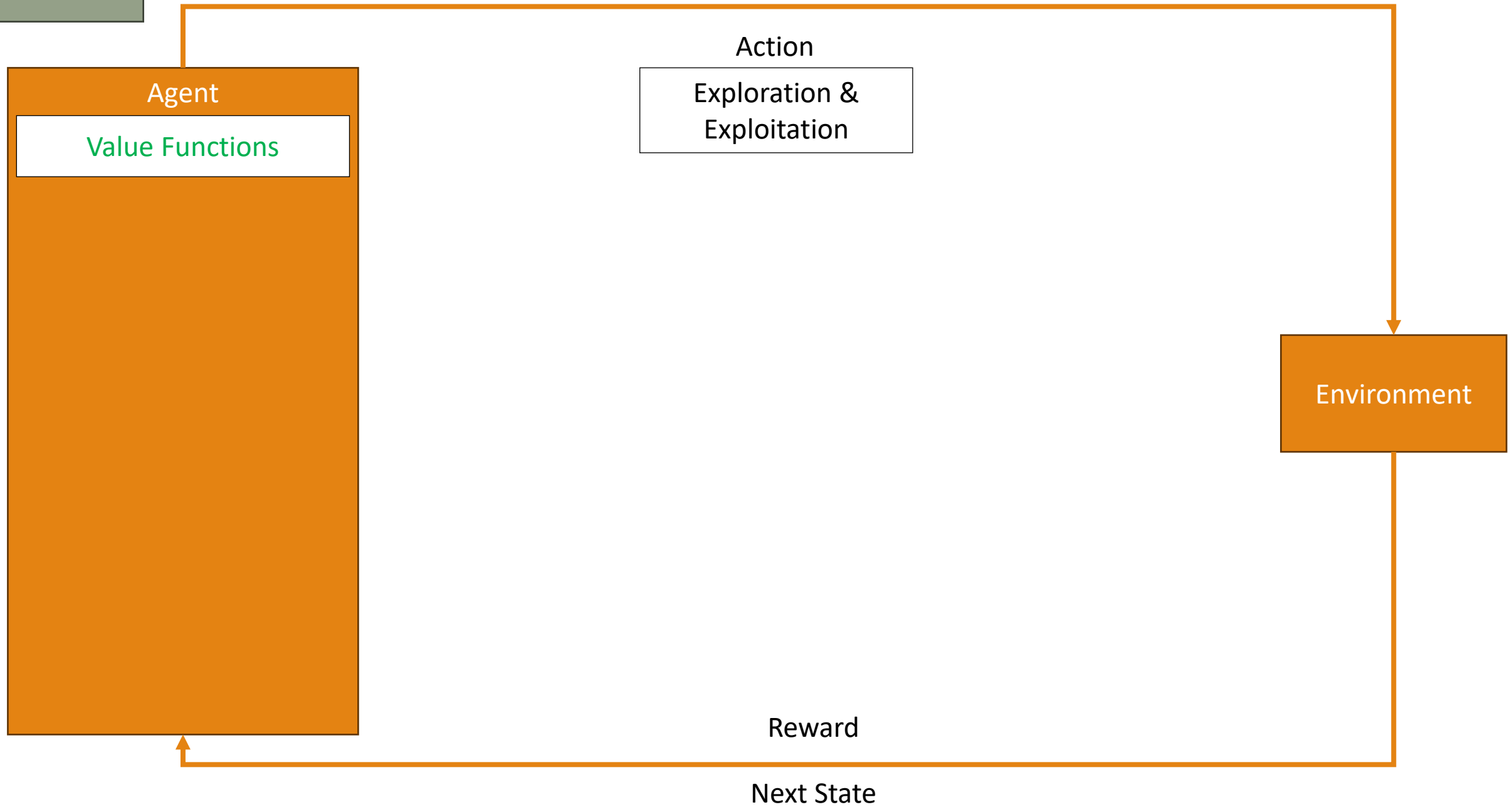
Lecture 1



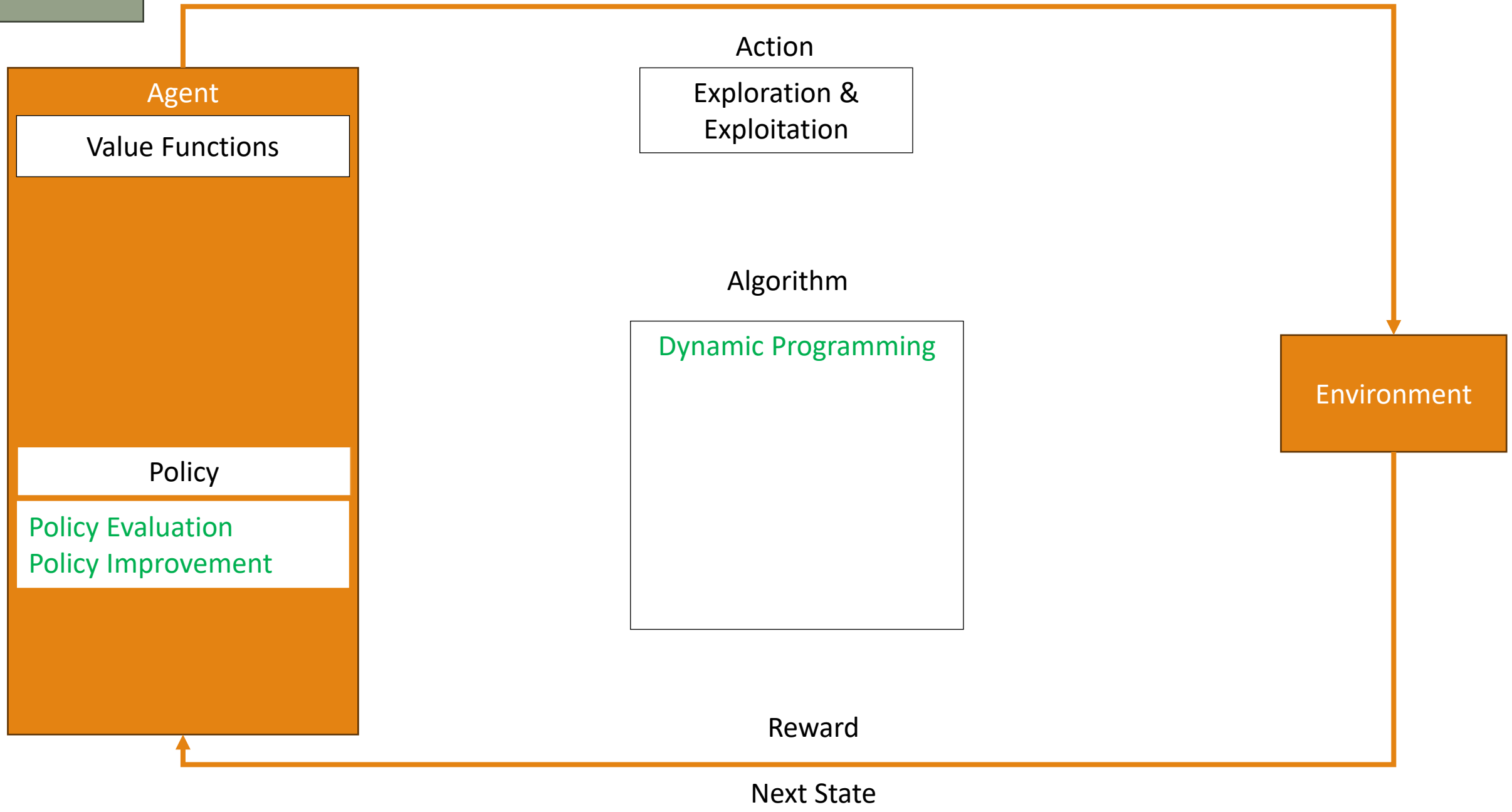
Lecture 2


Action
Exploration &
Exploitation
Agent
Environment
Reward
Next State

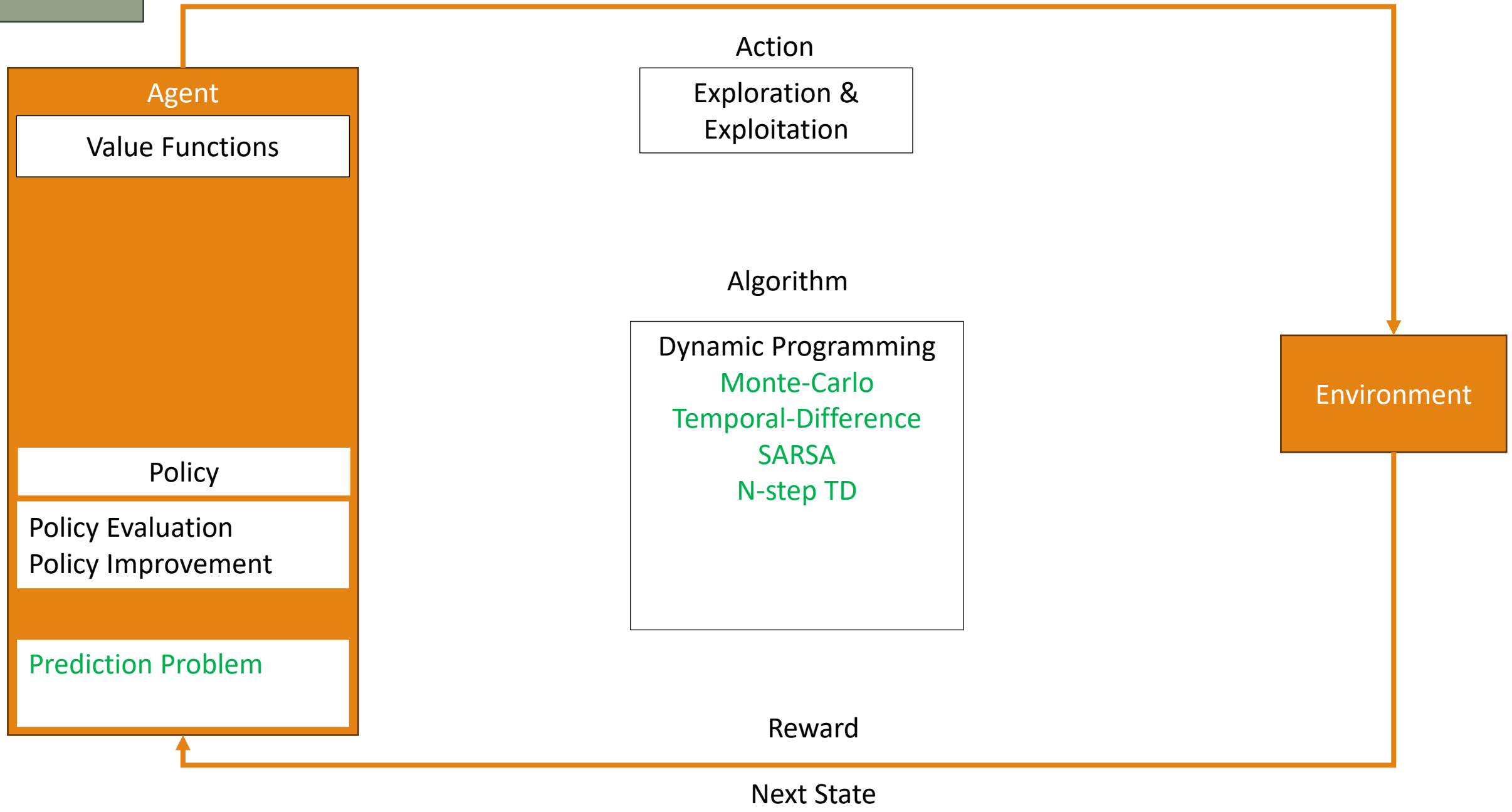
Lecture 3


Action
Exploration &
Exploitation
Agent
Value Functions
Environment
Reward
Next State

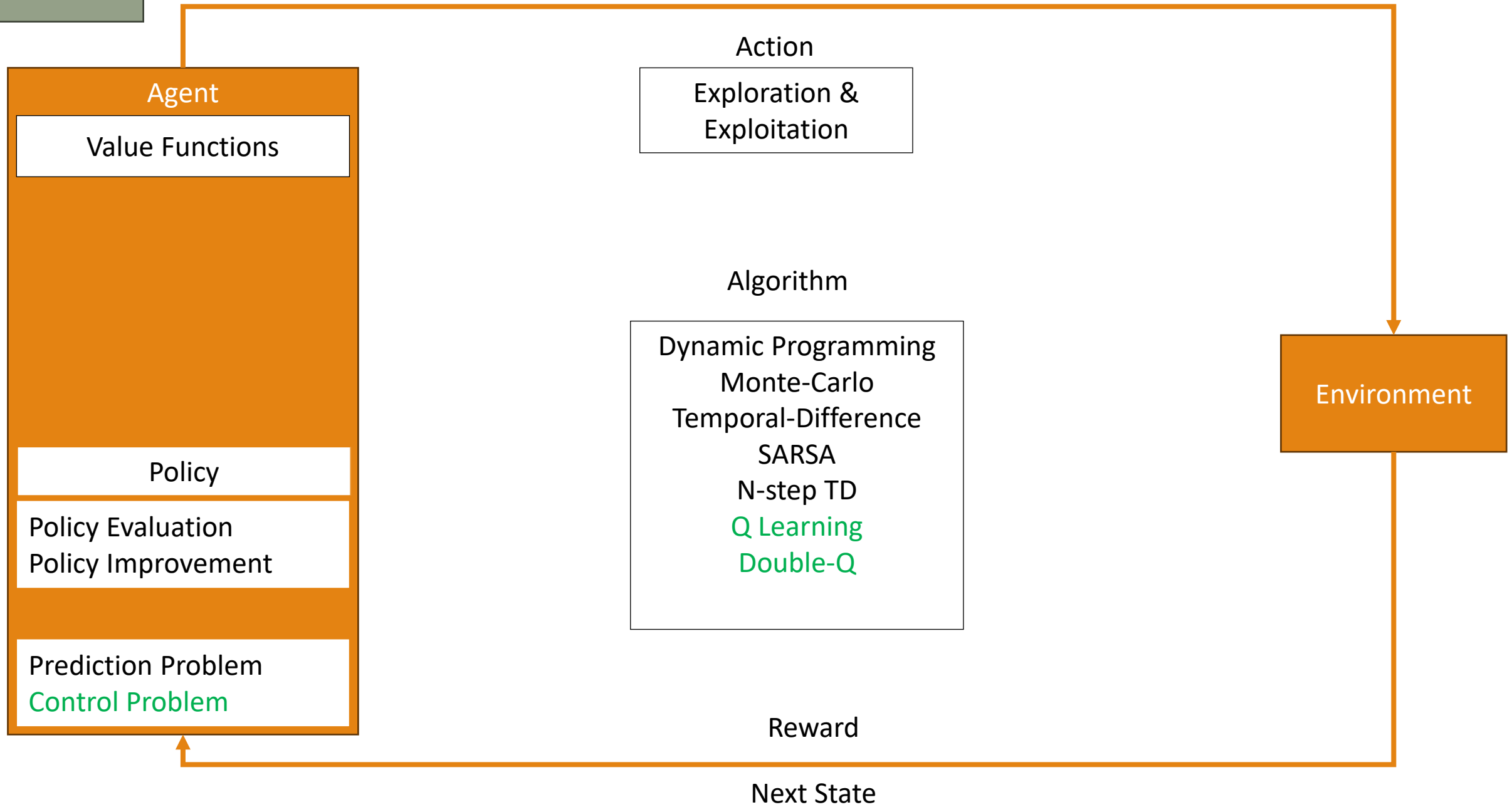
Lecture 4


Action
Exploration &
Exploitation
Agent
Value Functions
Algorithm
Dynamic Programming
Environment
Policy
Policy Evaluation
Policy Improvement
Reward
Next State

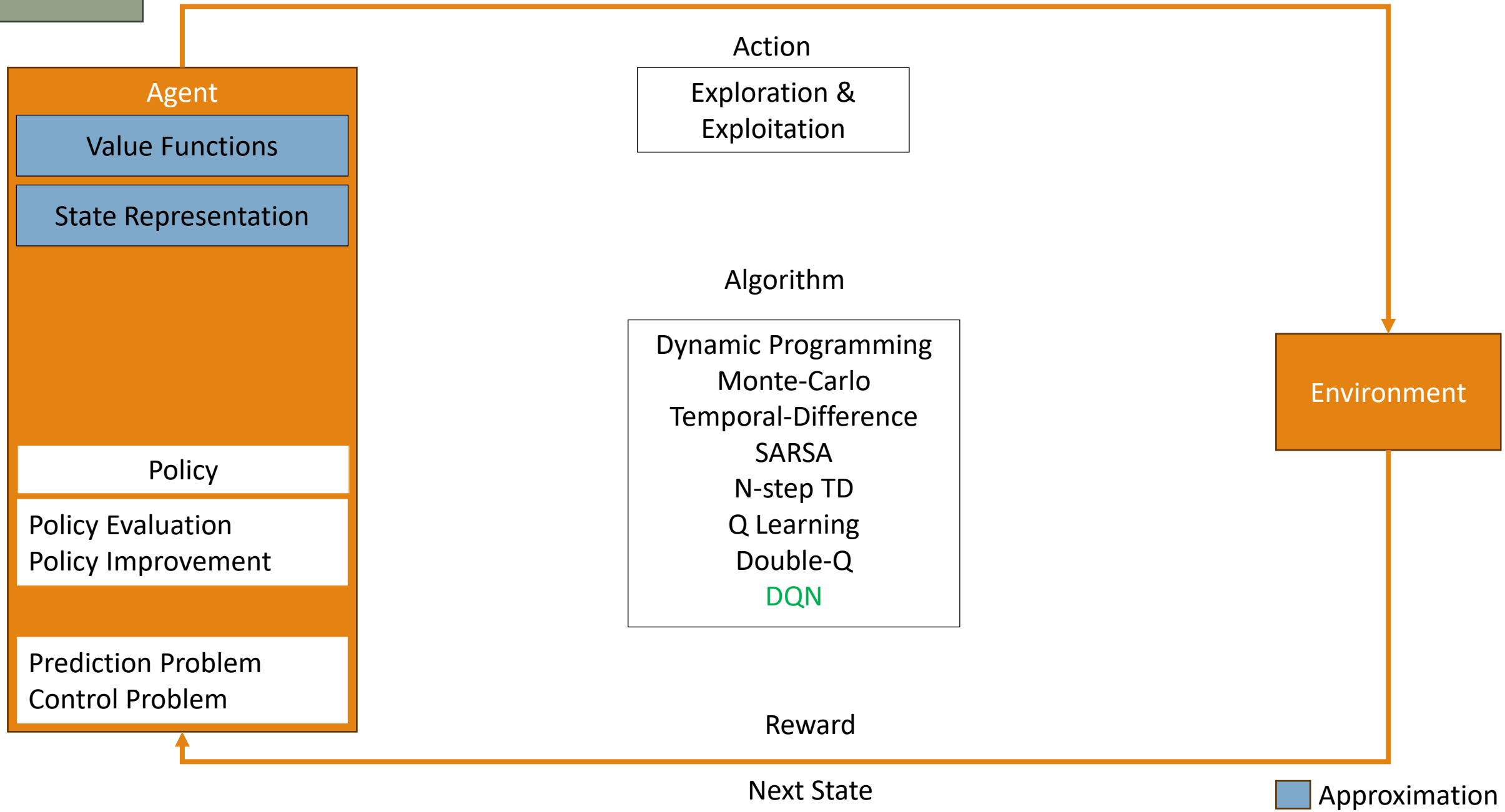
Lecture 5


Action
Exploration &
Exploitation
Agent
Value Functions
Algorithm
Dynamic Programming
Monte-Carlo
Temporal-Difference
SARSA
N-step TD
Environment
Policy
Policy Evaluation
Policy Improvement
Prediction Problem
Reward
Next State

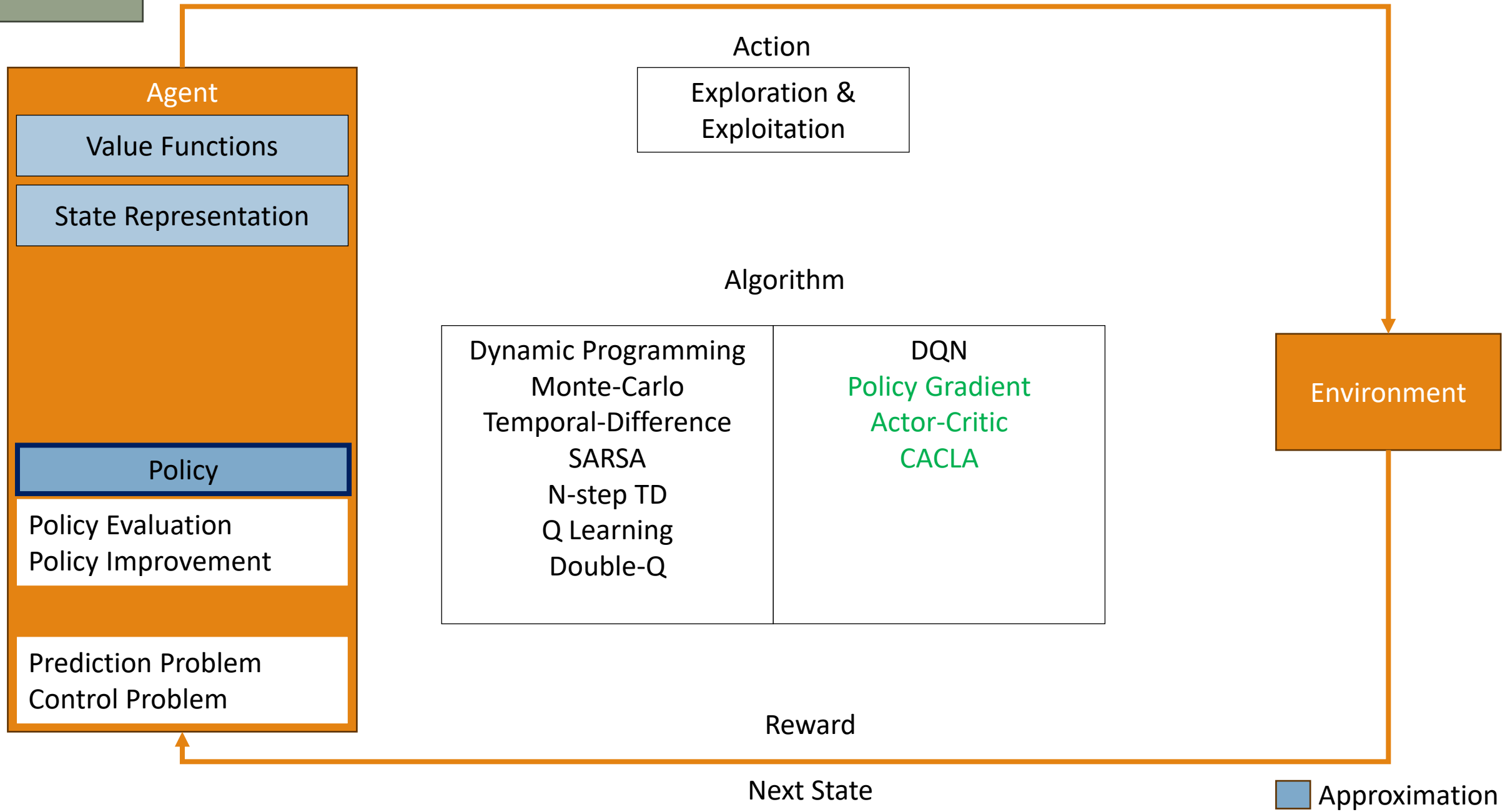
Lecture 6


Agent
Value Functions
Policy
Policy Evaluation
Policy Improvement
Prediction Problem
Control Problem
Action
Exploration &
Exploitation
Algorithm
Dynamic Programming
Monte-Carlo
Temporal-Difference
SARSA
N-step TD
Q Learning
Double-Q
Environment
Reward
Next State

Lecture 7


Agent
Value Functions
State Representation
Policy
Policy Evaluation
Policy Improvement
Prediction Problem
Control Problem
Action
Exploration &
Exploitation
Algorithm
Dynamic Programming
Monte-Carlo
Temporal-Difference
SARSA
N-step TD
Q Learning
Double-Q
DQN
Environment
Reward
Next State
Approximation

Lecture 7


Agent
Value Functions
State Representation
Policy
Policy Evaluation
Policy Improvement
Prediction Problem
Control Problem
Action
Exploration &
Exploitation
Algorithm
Dynamic Programming
Monte-Carlo
Temporal-Difference
SARSA
N-step TD
Q Learning
Double-Q
DQN
Policy Gradient
Actor-Critic
CACLA
Environment
Reward
Next State
Approximation

Today

Agent

Value Functions

State Representation

Model

Policy

Policy Evaluation
Policy Improvement

Prediction Problem
Control Problem

Action

Exploration &
Exploitation

Algorithm

Dynamic Programming Monte-Carlo Temporal-Difference SARSA N-step TD Q Learning Double-Q	DQN Policy Gradient Actor-Critic CACLA Dyna-Q

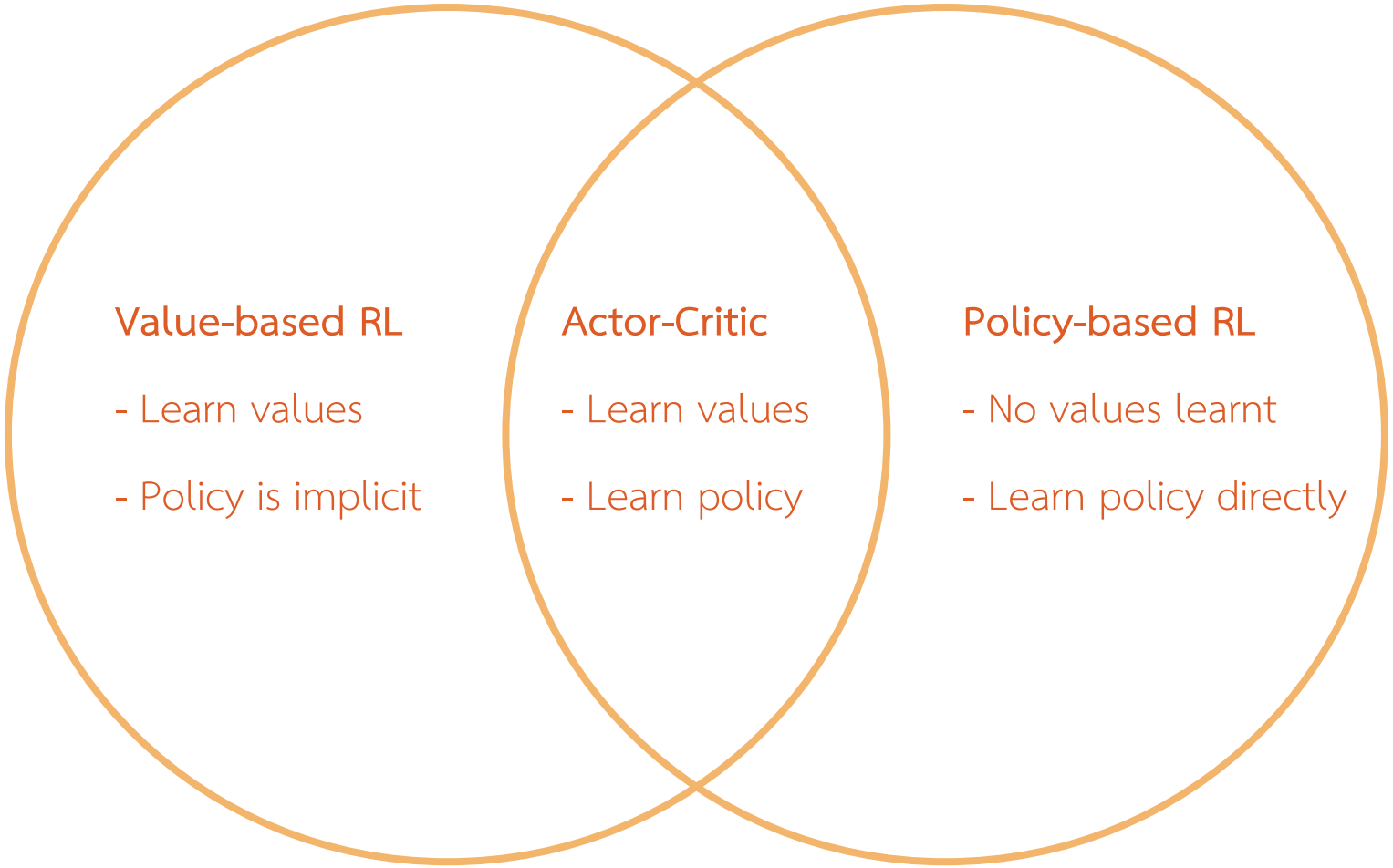
Environment

Reward

Next State

Approximation

Value-based and Policy-based RL



Planning and Models

Blink Sakulkueakulsuk

Model-based RL

Dynamic Programming

- Know the model
- **Solve** the model. No interaction needed!

Model-free RL

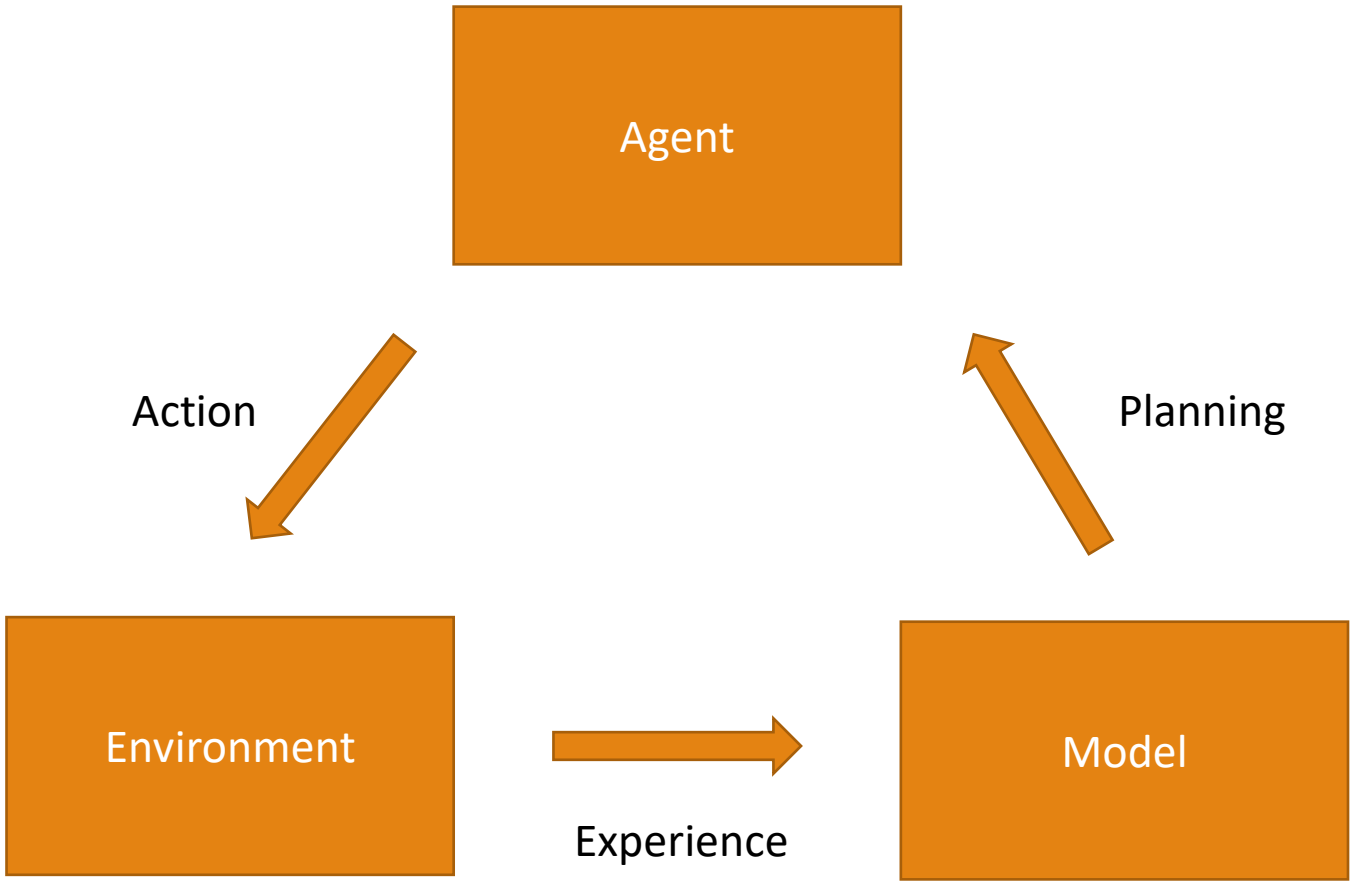
- No model
- **Learn value functions** from experiences

Model-based RL

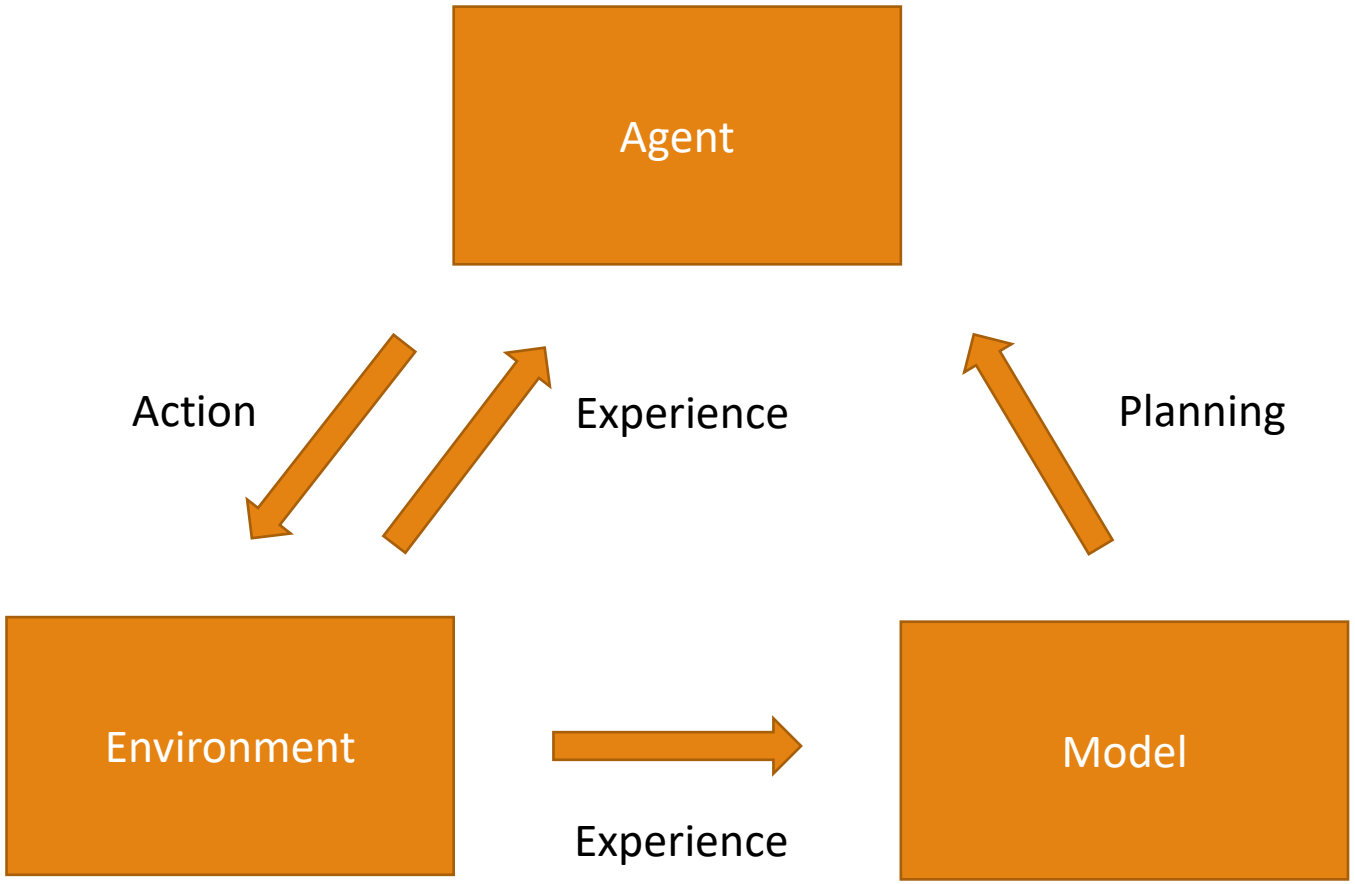
- **Learn a model** from experience
- **Plan value functions** from the model

Direct RL

Model-Based RL



Model-Based RL



Why model?

Approximation error is an obvious disadvantage for Model-based Reinforcement Learning

- We learn a model, then build a value function. There are two sources of approximation error

- Value-based RL approximate value function directly, so there are only one source of approximation error

Why model?

However

1. Models can be learned via supervised learning, which is a well-studied method
2. We can integrate uncertainty into the model
3. Reduce resources needed in learning (time and space efficiency)

A Model

A **model** $\boldsymbol{M_\eta}$ is a function approximation of an MDP $(S, A, \hat{p}_\eta)$.

Dynamics of the model $\hat{p}_\eta$ is parametrized by $\boldsymbol{\eta}$.

The model essentially is the approximation of state transitions and rewards, $\hat{p}_\eta \approx p$

$$R_{t+1}, S_{t+1} \sim \hat{p}_\eta(r, s' | S_t, A_t)$$

Learning the Model

The goal is to approximate a **model $\boldsymbol{M_\eta}$** from the experience

$$\{S_1, A_1, R_2, \ldots, S_T\}$$

We can formulate the experiences as a supervised learning problem

$$S_1, A_1 \rightarrow R_2, S_2$$

$$\ldots$$

$$S_{T-1}, A_{T-1} \rightarrow R_T, S_T$$

Learning the Model

To learn the model, we

1. Choose a class of function to approximate, $f_{\boldsymbol{\eta}}(s, a) = r, s'$
2. Set up loss function and/or objective function
3. Learn parameters $\boldsymbol{\eta}$ that minimize loss function

We will get **an expectation model**

- If $f_{\boldsymbol{\eta}}(s, a) = r, s'$, then $s' \approx \mathbb{E}[S_{t+1} | s = S_t, a = A_t]$

Expectation Models

Expectation models have some problems

- Because the model tries to learn the transitions from expectations, it may lead to a weird outcome
- Imagine a grid world with a wall ahead. An agent can either go left or right to avoid the wall, but the expectation will average out, resulted in an agent goes directly into a wall

Expectation Models

Expectation models still mostly represent linear value

Given an expectation model $f_{\boldsymbol{\eta}}(\phi_t) = \mathbb{E}[\phi_{t+1}]$, and a value function $v_{\boldsymbol{\theta}}(\phi_t) = \boldsymbol{\theta}^{\mathrm{T}}\phi_t$

$$\begin{aligned}\mathbb{E}[v_{\boldsymbol{\theta}}(\phi_{t+1}|S_t = s)] &= \mathbb{E}[\boldsymbol{\theta}^{\mathrm{T}}\phi_{t+1}|S_t = s] \\ &= \boldsymbol{\theta}^{\mathrm{T}}\mathbb{E}[\phi_{t+1}|S_t = s] \\ &= v_{\boldsymbol{\theta}}(\mathbb{E}[\phi_{t+1}|S_t = s])\end{aligned}$$

If the model is also linear $f_{\boldsymbol{\eta}}(\phi_t) = \mathbf{P}\phi_{t+1}$ for some matrix $\mathbf{P}$

- We can extend this model to approximate the future

$$\mathbb{E}[v_{\boldsymbol{\theta}}(\phi_{t+n}|S_t = s)] = v_{\boldsymbol{\theta}}(\mathbb{E}[\phi_{t+n}|S_t = s]$$

Stochastic Models

Many systems are non-linear, even noisy sometimes. In this case, expectation models cannot approximate accurately.

We can use **stochastic models** (aka **generative models**)

$$\hat{R}_{t+1}, \hat{S}_{t+1} = \hat{p}(S_t, A_t, \omega)$$

ω is a noise term

Full Models

We can try to model the complete system.

$$\mathbb{E}[v(S_{t+1})|S_t = s] = \sum_a \pi(a|s) \sum_{s'} \hat{p}(s,a,s')(\hat{r}(s,a,s') + \gamma v(s'))$$

This is hard to compute because of the branching of states and actions

Types of Models

Usually, we use different parametric functions to approximate the dynamics

- We can have one parametric function for transitions, and another for rewards

Each function can have different format

- Table Lookup Model
- Linear Expectation Model
- Linear Gaussian Model
- Deep Neural Network Model

Table Lookup Model

A table contains each state-action pair

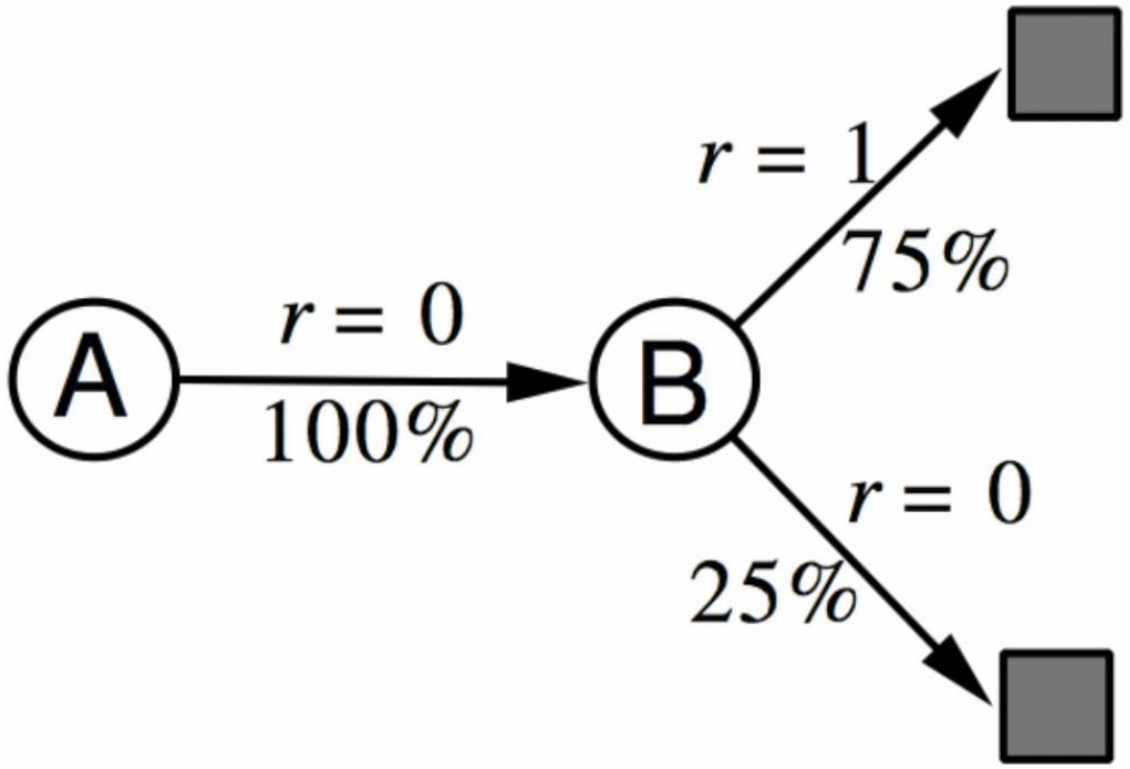
$$\hat{p}_t(s'|s,a) = \frac{1}{N(s,a)} \sum_{k=0}^{t-1} I(S_k = s, A_k = a, S_{k+1} = s')$$

$$\mathbb{E}_{\hat{p}_t}[R_{t+1}|S_t = s, A_t = a] = \frac{1}{N(s,a)} \sum_{k=0}^{t-1} I(S_k = s, A_k = a) R_{k+1}$$

AB Example

Given experiences:

A, 0, B, 0	B, 1
B, 1	B, 1
B, 1	B, 1
B, 1	B, 0



We can construct a look up table that give the diagram above.

Linear Expectation Model

We can use linear function as an expectation model. Given a feature representation vector $\phi(s)$,

State transition model is parametrized by a square matrix T

$$\hat{s}'(s,a) = \mathrm{T}_a \phi(s)$$

Rewards are parametrized by a vector $\mathbf{w}_a$

$$\hat{r}(s,a) = \mathbf{w}_a^T \phi(s)$$

On each transition, we can apply gradient descent to minimize loss

$$L(s,a,r,s') = \left(s' - \mathrm{T}_a \phi(s)\right)^2 + \left(r - \mathbf{w}_a^T \phi(s)\right)^2$$

Planning

To improve values and policy, we can **plan without interaction** with the environment

- Dynamic programming is an example of planning
- We will focus on algorithms that do not require for access to perfect environment
- We will plan using learned models

Sample-based Planning

Instead of planning from known model, we can **sample** the experience from a model

$$S, R \sim \hat{p}_\eta(\cdot | s, a)$$

We then apply model-free RL to learn from samples

- Monte-Carlo
- SARSA
- Q-learning

Planning with Inaccurate Models

If we have an inaccurate model,

- Planning may yield suboptimal policy
- Performance is limited to optimal policy for approximated MDP
- Model-based RL is as good as the estimated model

To deal with such problems, we can

- Use model-free if the model is wrong
- Introduce some uncertainty or noise into the model
- Utilize both model-based and model-free in a single algorithm

Source of Experiences

Consider two sources of experiences

Real experiences are sampled from the **environment**

$$r, s' \sim p$$

Simulated experience are sample from the **model**

$$r, s' \sim \hat{p}_\eta$$

Model-free and Model-based Recap

Model-free RL

- No model
- Learn value function (and/or policy) from real experience

Model-based RL

- Learn a model from real experience
- Plan value function (and/or policy) from simulated experience

Dyna

Combine both model-free and model-based algorithm together

- Learn a model from real experience
- **Learn and plan** value function (and/or policy) from real and simulated experience

Dyna-Q Algorithm

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Do forever:

(a) $s \leftarrow$ current (nonterminal) state

(b) $a \leftarrow \varepsilon$-greedy(s,Q)

(c) Execute action a; observe resultant state, s', and reward, r

(d) $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

(e) $Model(s,a) \leftarrow s', r$ (assuming deterministic environment)

(f) Repeat N times:

$s \leftarrow$ random previously observed state

$a \leftarrow$ random action previously taken in s

$s', r \leftarrow Model(s,a)$

$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

Advantages of Dyna

We can learn from updated model and past experience many times. This leads to efficiency in learning.

This is especially important when real-world task/data is

- expensive (e.g., robotics)
- slow
- unsafe (e.g., self-driving car)

Planning for Action Selection

So far, we learn to plan to improve a **global value** function.

However, learning involves changing a lot of distributions (values, policy, states, reward, ...), how should we select the next action?

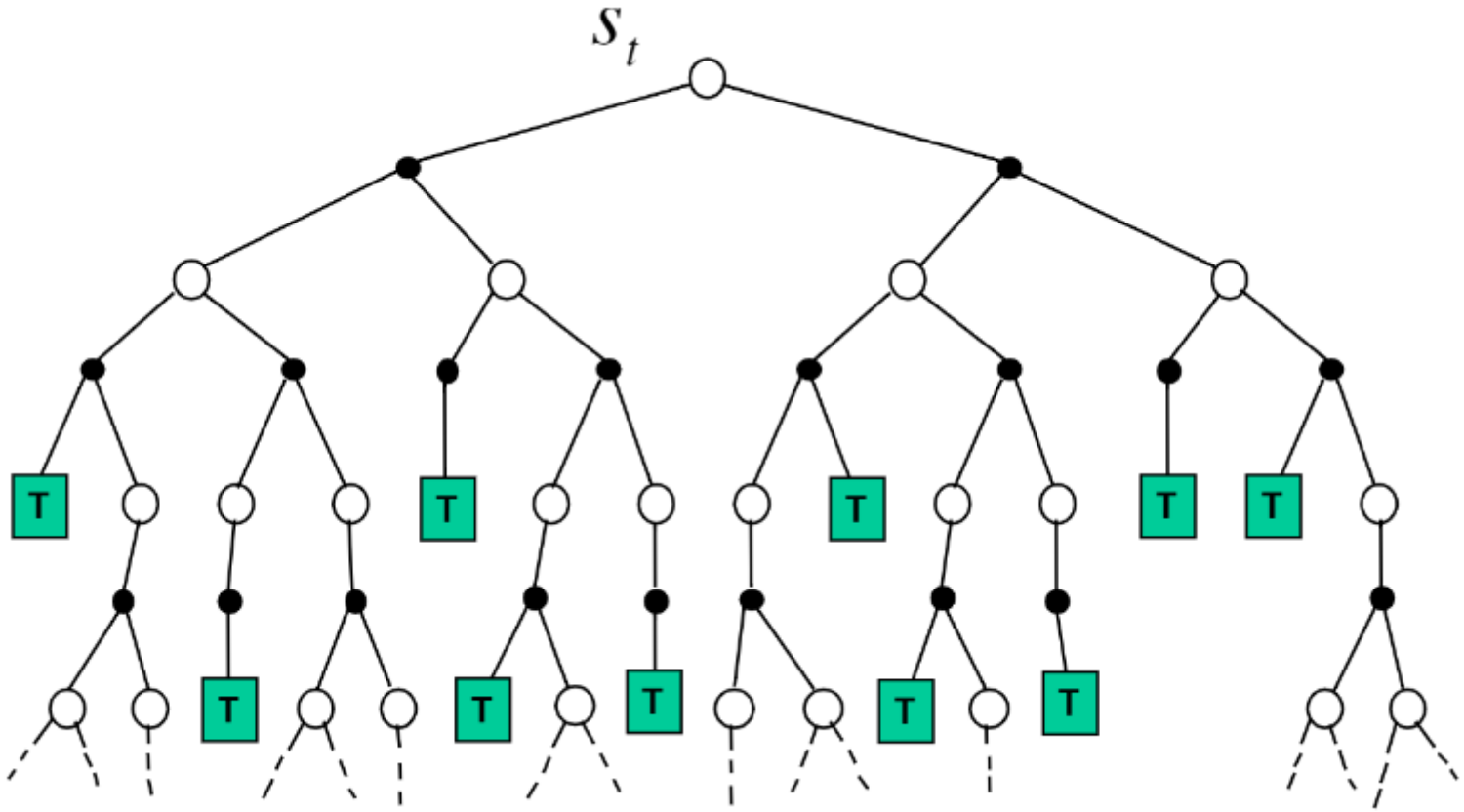
- Distribution at the start may differ from distribution now

- Should the agent focus on local value function rather than the global value function?

- Inaccuracies may open a way for more interesting exploration rather than bad updates

Forward Search

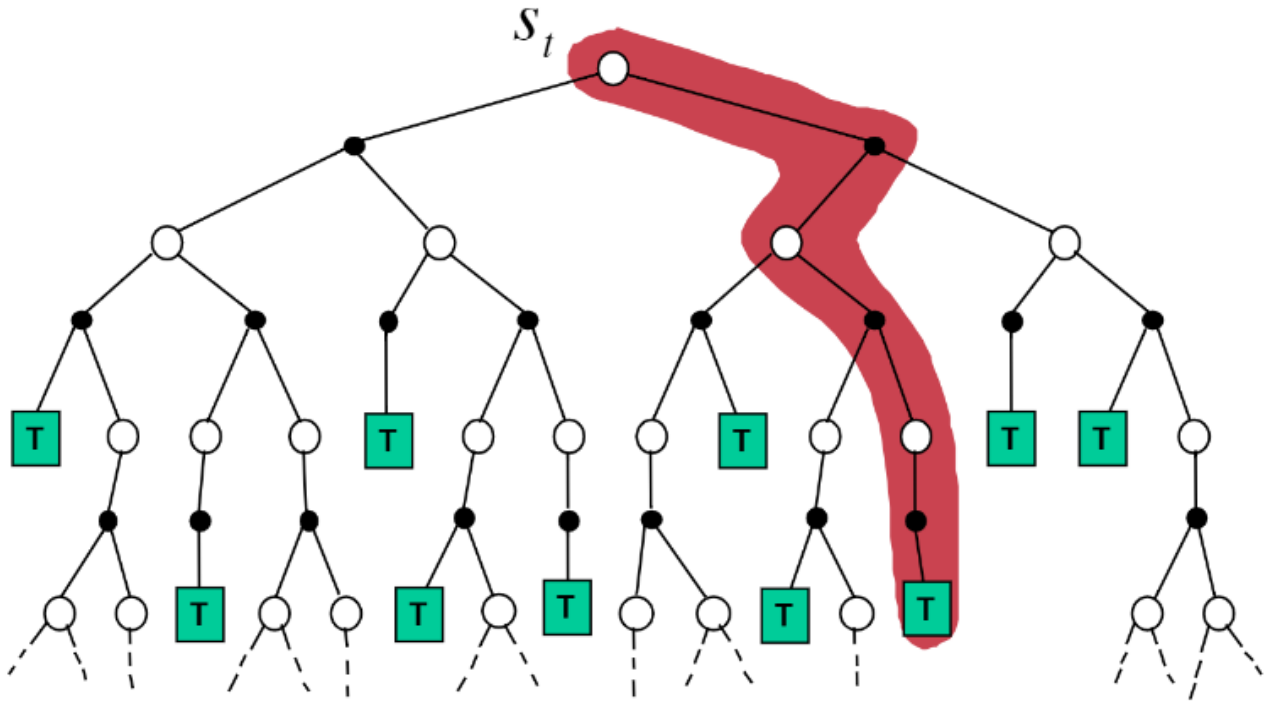
Build a search tree and use a model to look ahead



Simulation-based Search

Forward search using sample-based planning

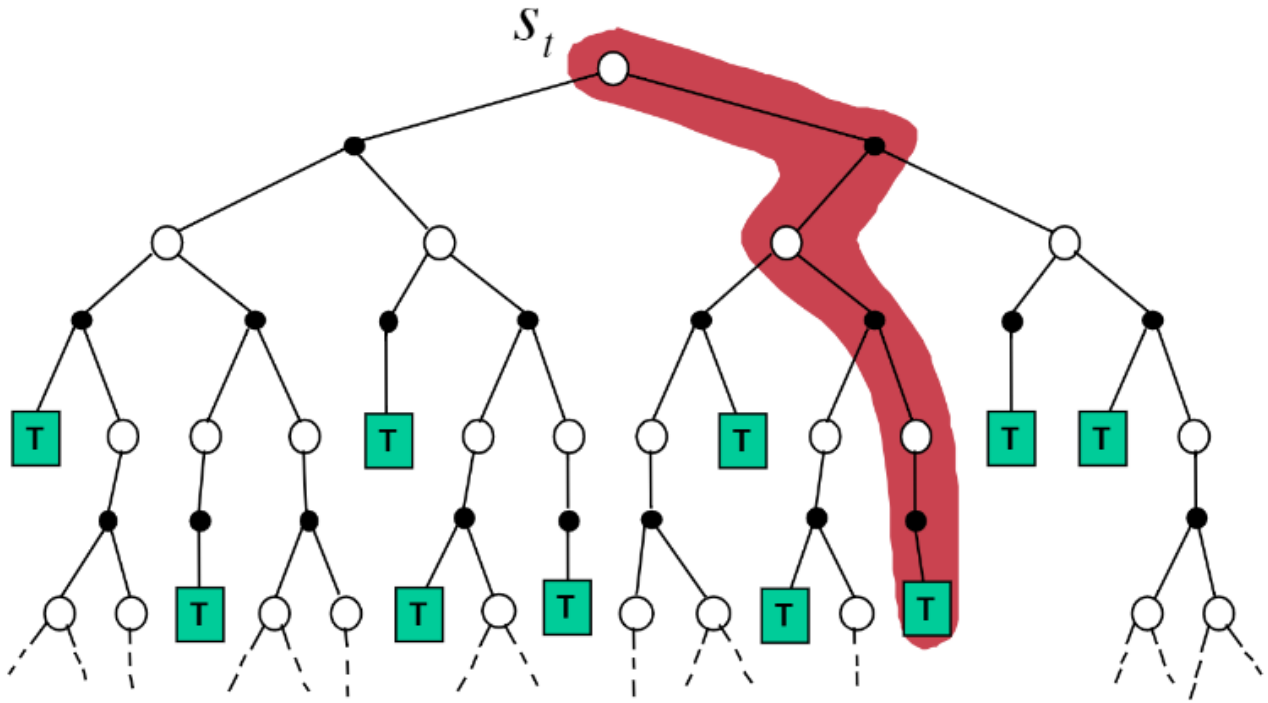
- Simulating episodes of experiences from now using the model
- Apply model-free RL to those episodes



Simulation-based Search

Simulating episodes of experiences from now using the model

$$\{S_t^k, A_t^k, R_{t+1}^k, \ldots, S_T^k\}_{k=1}^K \sim \hat{p}_\eta$$



Monte-Carlo Simulation

Given a parameterized model M_{η} and a **simulation** policy π

Simulate K episodes from current state S_t

$$\{S_t^k = S_t, A_t^k, R_{t+1}^k, \dots, S_T^k\}_{k=1}^K \sim \hat{p}_\eta, \pi$$

Evaluate the state by averaging returns of all episodes (Monte-Carlo evaluation)

$$v(S_t) = \frac{1}{K}\sum_{k=1}^{K} G_t^k$$

Simple Monte-Carlo Search

Given a parameterized model M_{η} and a **simulation** policy π

For each action $a \in A$

Simulate K episodes from current state S_t

$$\{\boldsymbol{S_t^k = s, A_t^k = a}, R_{t+1}^k, \dots, S_T^k\}_{k=1}^K \sim M_{\eta}, \pi$$

Evaluate state by averaging returns of all episodes

$$q(s,a) = \frac{1}{K}\sum_{k=1}^{K} G_t^k$$

Select action with maximum value

$$A_t = argmax_{a \in A} q(S_t, a)$$

Monte-Carlo Tree Search (Evaluation)

Given a parameterized model M_{η}

Simulate K episodes from current state S_t using a current simulation policy π

$$\{\boldsymbol{S_t^k = s}, A_t^k, R_{t+1}^k, \dots, S_T^k\}_{k=1}^K \sim M_{\eta}, \pi$$

Build a search tree containing visited states and actions

Evaluate states $q(s,a)$ by averaging returns of episodes from s, a

$$q(s,a) = \frac{1}{N(s,a)} \sum_{k=1}^{K} \sum_{u=t}^{T} 1(S_u^k, A_u^k = s,a) G_u^k$$

After searching, select action with maximum value

$$A_t = argmax_{a \in A} q(S_t, a)$$

Monte-Carlo Tree Search (Simulation)

In MCTS, the simulation policy π improves

- Tree policy (improves): pick actions from $q(s,a)$ (e.g. $\epsilon - greedy$)
- Rollout policy (fixed): random actions

Repeat for each simulated episode

- Select actions in tree according to tree policy
- Expand search tree by one node
- Rollout to termination with default policy
- Update action-values in the tree

Output best actions when time runs out

Movie Time

AlphaGo – The Movie

https://www.youtube.com/watch?v=WXuK6gekU1Y&ab_channel=GoogleDeepMind